

Mentoría M10

Práctico 2: Análisis Exploratorio, Curación de Datos e Ingeniería ETL

SalarIA - Cobertura terrestre en ambientes salares

Entrega 04/09

Objetivo: transformar un dataset tabular de observaciones satelitales en un dataset depurado, documentado y listo para aprendizaje automático.

Índice

1. Sentido del Práctico 2	3
2. Estructura conceptual del dataset	3
3. De EDA a curación	3
4. Pipeline ETL reproducible	3
4.1. Extract	4
4.2. Load a staging	4
4.3. Transform	4
5. Contrato de datos	4
6. Valores faltantes	4
6.1. Políticas posibles	5
7. Duplicados	5
8. Categorías inconsistentes	5
9. Outliers univariados	5
10. Outliers multivariados	6
11. Redundancia y correlación	6
12. Separabilidad estadística por clase	6
13. Balance de clases	6
14. Ingeniería geoespacial	7
15. Variables que no deben entrar como predictoras	7
16. Bitácora de decisiones	7

17.Pipeline para ML	8
18.Criterios de evaluación de una buena entrega	8
19.Dimensiones de calidad de datos	8
20.Métodos estadísticos recomendados	9
20.1. Asimetría y curtosis	9
20.2. Pruebas de diferencia entre clases	9
20.3. Tamaño de efecto	9
20.4. Información mutua	9
21.Outliers: decisiones posibles	9
22.Eschema SQLite sugerido	9
23.Consultas SQL útiles para auditar	10
24.Comparación con dataset limpio de referencia	10
25.Errores frecuentes en el Práctico 2	10
26.Rúbrica sugerida	11
27.Conclusión	11

1. Sentido del Práctico 2

El Práctico 1 permitió comprender el dataset, visualizar distribuciones, comparar clases y formular hipótesis. El Práctico 2 exige un salto metodológico: pasar de mirar los datos a **tomar decisiones de curación reproducibles**. En ciencia de datos aplicada a ambiente, una limpieza apresurada puede destruir señales reales. Por eso, la curación debe combinar estadística, conocimiento del dominio y trazabilidad.

Idea central

Un outlier no es necesariamente un error. En ambientes salares pueden existir respuestas espectrales extremas por humedad, costras salinas, suelo desnudo, roca, sombras, mezcla de píxeles o variabilidad mineralógica. La decisión correcta no es borrar automáticamente, sino auditar, justificar y documentar.

2. Estructura conceptual del dataset

Cada fila representa una observación tabular asociada a un píxel extraído de un polígono de referencia. Las columnas pueden organizarse en seis familias:

Grupo	Ejemplos	Uso
Identificación	<code>sample_id</code>	Trazabilidad de la observación y prevención de fuga de información.
Etiquetas	<code>class_id</code> , <code>class_name</code> , <code>macro_class</code>	Variable objetivo o metainformación de clase. No deben entrar como predictoras.
Metadatos	<code>season</code> , <code>confidence</code> , <code>split</code> , <code>source</code> , <code>notes</code>	Control, auditoría y trazabilidad.
Bandas	B02, B03, B04, B08, B11, B12	Respuesta espectral Sentinel-2.
Índices	NDVI, NDWI, MNDWI, BSI	Variables derivadas para separar coberturas.
Espaciales	<code>x</code> , <code>y</code> , <code>scl</code>	Ubicación y control de calidad de escena.

3. De EDA a curación

El análisis exploratorio describe. La curación decide. Una buena entrega debe mostrar el puente entre ambos:

1. Diagnóstico: qué problema se detectó.
2. Evidencia: cuántas filas, columnas o clases afecta.
3. Decisión: eliminar, imputar, transformar, marcar o conservar.
4. Justificación: estadística, técnica y de dominio.
5. Resultado: dataset final, log de decisiones y reporte reproducible.

4. Pipeline ETL reproducible

ETL significa **Extract, Transform, Load**. Para este práctico se propone:

CSV originales → staging SQLite → quality tables → curated mart →
CSV/Parquet final

4.1. Extract

Se leen los archivos originales sin modificarlos. Debe existir una función única de carga que funcione localmente o desde GitHub.

```
from pathlib import Path
import pandas as pd

RAW_BASE = "https://raw.githubusercontent.com/pablonicolasr/mentodatos_pdis/main"

def read_dataset(relative_path):
    local = Path(relative_path)
    if local.exists():
        return pd.read_csv(local)
    return pd.read_csv(f"{RAW_BASE}/{relative_path}")

df = read_dataset("tables/olaroz_samples_features.csv")
```

4.2. Load a staging

SQLite permite guardar las tablas de entrada y los reportes de calidad en un archivo portable.

```
from sqlalchemy import create_engine
engine = create_engine("sqlite:///data/warehouse/m10_practico2_etl.sqlite")
df.to_sql("stg_olaroz_samples_features", engine, if_exists="replace", index=False)
```

4.3. Transform

La transformación incluye normalización de nombres, reemplazo de infinitos, imputación si corresponde, flags de outliers, variables derivadas y exclusión de columnas con fuga.

5. Contrato de datos

Un contrato de datos define condiciones mínimas:

- columnas requeridas: `sample_id`, `class_name`;
- columnas numéricas: bandas, índices y coordenadas;
- dominio esperado de `split`: train, validation, test;
- consistencia `sample_id` → clase;
- ausencia de infinitos;
- duplicados exactos y duplicados semánticos controlados;
- rangos físicamente plausibles para reflectancias e índices.

6. Valores faltantes

Los faltantes se analizan por columna, por fila y por patrón. No alcanza con ejecutar `df.isna().sum()`. Hay que responder:

1. ¿Cuánto falta?
2. ¿Dónde falta?

3. ¿Falta al azar o asociado a una clase/split?
4. ¿Conviene imputar o eliminar?
5. ¿La imputación debe aprenderse sólo en train?

6.1. Políticas posibles

Situación	Decisión sugerida	Justificación
Sin faltantes	No imputar	No introducir transformación innecesaria.
Faltantes bajos en numéricas	Mediana	Robusta ante asimetría.
Faltantes bajos en categóricas	Moda o categoría desconocido	Mantiene filas y explicita ausencia.
Faltantes estructurales	Crear flag de faltante	La ausencia puede contener información.
Columna con faltantes masivos	Evaluar descarte	Puede ser poco confiable.

7. Duplicados

Hay dos tipos:

- **duplicados exactos:** toda la fila se repite;
- **duplicados semánticos:** se repite una clave como `sample_id + x + y + class_name`.

Los exactos suelen eliminarse. Los semánticos deben auditarse porque pueden revelar problemas de extracción, mezcla de etapas o registros incompatibles.

8. Categorías inconsistentes

En datos reales aparecen problemas como espacios, mayúsculas, tildes, sinónimos o etiquetas mal escritas. La limpieza mínima incluye `strip`, normalización de texto y verificación de dominios.

9. Outliers univariados

Métodos recomendados:

- IQR: $[Q1 - 1,5IQR, Q3 + 1,5IQR]$.
- Z-score: $|z| > 3$.
- MAD robusto: $|x - mediana|/MAD > 3,5$.

Advertencia

En el dataset del salario, los outliers deben marcarse y revisarse antes de eliminarse. Una firma extrema de agua, sal o roca puede ser una señal válida.

10. Outliers multivariados

Puede ocurrir que una variable no sea extrema por separado, pero la combinación sí sea rara. Para eso se propone Isolation Forest como diagnóstico:

```
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler

X = df[feature_cols].replace([np.inf, -np.inf], np.nan)
X = X.fillna(X.median())
Xs = StandardScaler().fit_transform(X)
iso = IsolationForest(n_estimators=300, random_state=42)
df["flag_outlier_iso"] = iso.fit_predict(Xs) == -1
```

11. Redundancia y correlación

Bandas cercanas pueden estar altamente correlacionadas. Los índices derivados también pueden duplicar información. La redundancia no siempre es mala, pero debe medirse.

- Correlación Pearson: relación lineal.
- Correlación Spearman: relación monótona, más robusta.
- Pares con $|r| > 0,95$: candidatos a revisión.
- VIF: inflación de varianza por multicolinealidad.

12. Separabilidad estadística por clase

Para evaluar variables útiles se recomienda combinar:

- ANOVA: diferencia de medias entre clases;
- Kruskal-Wallis: alternativa no paramétrica;
- eta cuadrado: tamaño del efecto;
- información mutua: dependencia no necesariamente lineal;
- boxplots e histogramas por clase como apoyo visual.

Una variable no debe elegirse sólo por p-valor. En datasets grandes, diferencias pequeñas pueden ser significativas. Por eso se mira también el tamaño de efecto.

13. Balance de clases

El balance debe analizarse por píxel y por polígono. Si una clase tiene pocos polígonos pero muchos píxeles, el modelo puede parecer robusto aunque haya poca diversidad real.

Regla práctica

Para dividir train/test, priorizar `sample_id` como grupo. Evitar que píxeles del mismo polígono aparezcan simultáneamente en entrenamiento y evaluación.

14. Ingeniería geoespacial

Con x e y pueden generarse:

- coordenadas centradas;
- distancia al centroide global;
- ángulo respecto del centroide;
- distancia al centroide del polígono;
- densidad local aproximada;
- si hay geometrías: área, perímetro y compacidad.

Estas variables son útiles, pero deben usarse con cuidado: pueden capturar ubicación más que propiedades espectrales y sobreajustar al área de estudio.

15. Variables que no deben entrar como predictoras

Variable	Motivo
<code>class_id</code> , <code>class_name</code> , <code>macro_class</code>	Son etiqueta o derivados directos de la etiqueta.
<code>sample_id</code>	Identificador de grupo; puede inducir fuga.
<code>split</code>	Define partición; no es una característica ambiental.
<code>source</code> , <code>notes</code>	Metadatos de origen o documentación.
<code>season</code> , <code>confidence</code> si son constantes	No aportan varianza predictiva.
<code>scl</code>	Conviene tratarlo como control de calidad, no como predictor principal.

16. Bitácora de decisiones

La entrega debe incluir una tabla como esta:

Problema	Evidencia	Decisión	Justificación
Nulos	0 celdas	No imputar	No introducir cambios innecesarios.
Infinitos	n detectados	Reemplazar por NaN + imputar	Evita errores numéricos.
Duplicados	n filas	Eliminar exactos	Sesgan frecuencias.
Outliers	n flags	Marcar, no borrar	Pueden ser ambientalmente válidos.
Correlación alta	pares $> 0,95 $	Revisar o seleccionar	Reducir redundancia.
Fuga	múltiples píxeles por sample	Split por grupo	Evaluación más honesta.

17. Pipeline para ML

El resultado del Práctico 2 debe conectar directamente con el Práctico 3:

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

preprocess = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]), numeric_features)
])
```

18. Criterios de evaluación de una buena entrega

1. El notebook corre de punta a punta.
2. Los datos originales no se modifican.
3. Existe un dataset curado con nombre nuevo.
4. La curación está justificada con evidencia.
5. Hay controles estadísticos y de dominio.
6. Hay prevención explícita de fuga por `sample_id`.
7. Hay bitácora de decisiones.
8. Hay salida reproducible: CSV, SQLite, reporte y entorno.

19. Dimensiones de calidad de datos

Una auditoría madura no se limita a una lista de errores. Evalúa dimensiones complementarias:

Dimensión	Pregunta	Métrica posible
Compleitud	¿Faltan datos?	Porcentaje de nulos por columna y por fila.
Validez	¿Los valores respetan dominios?	Rangos, tipos, categorías permitidas.
Consistencia	¿Las relaciones internas se cumplen?	<code>sample_id -> class_name;</code> <code>class_id -> class_name.</code>
Unicidad	¿Hay duplicados?	Duplicados exactos y claves semánticas repetidas.
Representatividad	¿Las clases están balanceadas?	Conteo por clase, por polígono y por split.
Trazabilidad	¿Se puede reconstruir el proceso?	SQLite, decision log, metadatos del run.
Utilidad	¿Sirve para modelar?	Separabilidad, redundancia, información mutua.

20. Métodos estadísticos recomendados

20.1. Asimetría y curtosis

La asimetría mide si la distribución se concentra hacia un lado. La curtosis ayuda a detectar colas pesadas. En bandas e índices de salares son esperables distribuciones no normales, por lo que conviene acompañar media y desvío con mediana, cuantiles y gráficos.

20.2. Pruebas de diferencia entre clases

Si una variable separa clases, las distribuciones deberían diferir. Dos pruebas útiles son:

- **ANOVA**: compara medias entre clases. Es útil como diagnóstico, aunque asume condiciones que pueden no cumplirse.
- **Kruskal-Wallis**: compara rangos y es más robusta cuando las distribuciones son asimétricas o no normales.

20.3. Tamaño de efecto

El p-valor no alcanza. En datasets grandes casi todo puede resultar significativo. Por eso conviene reportar un tamaño de efecto, por ejemplo:

$$\eta^2 = \frac{SS_{between}}{SS_{total}}$$

Una variable con p-valor pequeño pero η^2 muy bajo puede no ser tan útil para modelado.

20.4. Información mutua

La información mutua mide dependencia entre una variable y la clase sin exigir linealidad. Es especialmente útil cuando un índice separa una clase sólo en ciertos rangos.

21. Outliers: decisiones posibles

Acción	Cuándo usarla	Riesgo
Conservar	Valor extremo plausible	El modelo puede verse afectado si es ruido.
Marcar con flag	Hay duda razonable	Aumenta columnas, pero conserva información.
Winsorizar	Extremos numéricos muy influyentes	Puede recortar señales reales de sal/agua.
Imputar	Valor inválido o imposible	Debe hacerse dentro del pipeline, no antes del split.
Eliminar fila	Error evidente o duplicado	Puede reducir clases minoritarias.

22. Esquema SQLite sugerido

Tabla	Propósito
<code>stg_pixels_raw</code>	Copia exacta de datos brutos tabulares.
<code>stg_pixels</code>	Datos luego de controles iniciales.

stg_features	Dataset enriquecido recomendado para curación.
quality_profile_variables	Perfil por variable: nulos, únicos, cuantiles, asimetría.
quality_integrity_summary	Resumen por dataset: filas, duplicados, infinitos.
statistics_feature_ranking	Ranking de variables por información mutua y F-score.
curation_decision_log	Decisiones de curación.
mart_olaroz_samples_clean	Tabla final curada para ML.

23. Consultas SQL útiles para auditar

```
import pandas as pd
from sqlalchemy import create_engine, text

engine = create_engine("sqlite:///data/warehouse/m10_practico2_etl.sqlite")

# Conteo por clase
pd.read_sql("""
SELECT class_name, COUNT(*) AS n
FROM stg_features
GROUP BY class_name
ORDER BY n DESC
""", engine)

# Duplicados por clave semantica
pd.read_sql("""
SELECT sample_id, x, y, class_name, COUNT(*) AS n
FROM stg_features
GROUP BY sample_id, x, y, class_name
HAVING COUNT(*) > 1
""", engine)
```

24. Comparación con dataset limpio de referencia

El archivo `olaroz_samples_clean.csv` puede usarse como referencia, pero no debe reemplazar la curación del grupo. Una comparación útil es:

- diferencias de filas;
- diferencias de columnas;
- clases presentes;
- variables eliminadas/agregadas;
- métricas de calidad antes/después;
- impacto de la curación sobre el balance de clases.

25. Errores frecuentes en el Práctico 2

1. Borrar outliers sólo porque aparecen fuera del boxplot.
2. Usar `class_id` o `class_name` como predictor.
3. Separar train/test por fila y no por `sample_id`.
4. Hacer imputación antes de separar los datos de entrenamiento.

5. No guardar el dataset curado.
6. No documentar decisiones.
7. Hacer gráficos sin concluir qué decisión provocan.
8. Modificar los CSV originales del repositorio.

26. Rúbrica sugerida

Criterio	Peso	Evidencia esperada
Reproducibilidad	20 %	Notebook corre completo, rutas relativas, entorno documentado.
EDA orientado a curación	20 %	Gráficos y estadísticas terminan en decisiones.
Calidad y estadística	25 %	Nulos, duplicados, outliers, correlación, balance y separabilidad.
ETL e ingeniería de datos	20 %	SQLite/SQLAlchemy, staging, quality tables, mart final.
Comunicación técnica	15 %	Reporte claro, decision log y conclusiones defendibles.

27. Conclusión

La curación de datos no es una etapa menor. En este proyecto, define si los modelos futuros aprenderán propiedades de las coberturas terrestres o simplemente memorizarán artefactos de extracción, ubicación o partición. La meta del Práctico 2 es dejar un dataset que pueda defenderse técnica y científicamente.